Effects of Parameter Averaging and Model Ensembling on the Accuracy of Food Image Classification

1st Mahiv Khan Wasi
1931673042
North South University

2nd Sibghat Ullah Rayyan Shaikh
1831773042
North South University

3rd Ahmad Hossain
1931259042
North South University

4th Rashidul Hassan Borshon
1931801642
North South University

***Abstract*—Food is essential for our lives. With the onset of the internet, every day thousands of photos of food are shared daily over social media platforms worldwide. From individuals posting photos of their meals to food bloggers and critics sharing their images on their websites and social media to a global audience, food images make up a vast portion of media found on the internet. Therefore correct identification of differences is essential for advertisers looking to advertise their content based on individual preference. The wrong classification of food images results in advertisers missing out on their target audience as well as health-conscious people being unable to check important details like calorie intake and ingredients. Nowadays, AI and image recognition are making it easier to correctly identify food images, with researchers finding various methods to do this more accurately and efficiently. In our experiment, we tried to develop a technique that is able to provide more accurate results using our limited resources and models. We experimented with techniques like model ensemble and parameter averaging to get a list of values showing accuracy. We trained 74 models and 15 different combinations. While training single models and averaging the parameter, accuracy increased in 75% of the models. Furthermore, 11 out of 15 combinations had a greater accuracy, including the averaging parameter technique. Hence 73.33% combinations had improved accuracy. The combination of InceptionV3, Resnet50, and VGG16 models using the average ensemble technique and average parameters had the highest accuracy of 95.24%.**

***Index Terms*—Model Ensemble, Food Image, Food-11, Classification, Parameter Averaging**

I. Introduction

Food is a vital part of our daily life. Every day, thousands of pictures are shared over social media platforms across the globe. This phenomenon was bombarded after smartphone took over the world as it has integrated cameras so you can click any picture anytime. People love sharing images of their meals with friends and family over various social media platforms. Food bloggers and food critics also share food images on their websites and blogs, and these images can be seen by a global audience. This is why food image classification using machine learning is important for advertisers as well as health-conscious people.

Advertisers look to advertise their content in the interest of one's individual interests. Often, they miss out on detecting the pictures shared by their clients which results in poor and ineffective advertisements. People who are health conscious miss out on important details of calorie intake because often many food images are not recognized properly. Artificial intelligence and image recognition are making it much simpler to identify an object from an image even if it is not labeled correctly. Many researchers have taken different initiatives to make a robust system using deep learning for food image classification[]. Techniques for image processing and computer vision are currently used in a wide range of fields. Food recognition from images is one of the domains. Food recognition is a difficult challenge because food images have less intraclass variation. Food recognition is important for a variety of applications:

A. *Nutrition tracking*

With food image classification, you can take a photo of your meals and use an app to automatically track your nutrient intake.

B. *Recipe recommendation*

A food image classifier can be used to recommend recipes based on the ingredients that are present in a photo of your pantry or fridge.

C. *Food waste reduction*

By being able to accurately classify food items, it is possible to develop systems that help people reduce food waste by reminding them to use perishable items before they expire.

D. *Menu design*

Food image classification can be used to help restaurants and other food service establishments design visually appealing menus that are more likely to attract customers.

E. *Food safety*

Image classification can be used to identify potentially hazardous food items, such as ones that are past their expiration date or have been contaminated.

In our paper, we have taken four pre-trained models (Alexnet, VGG-16, InceptionV3, ResNet50) and conducted several experiments. We ensembled the four different models in every combination possible. Our goal was to show that averaging parameters of the pre-trained models before ensembling them, increases accuracy. Various methods have been used

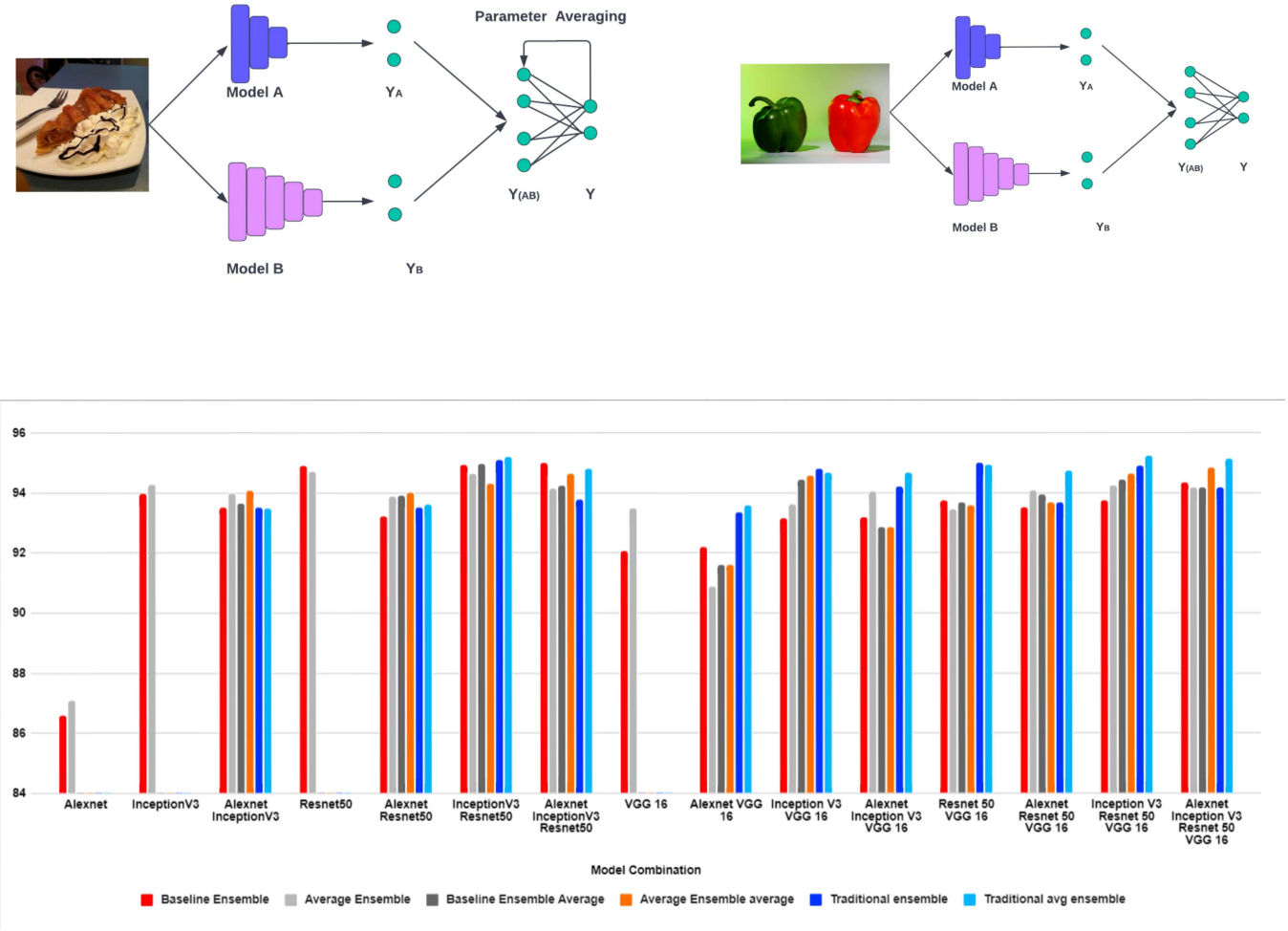


Fig. 1. Effects of Parameter Averaging and Model Ensembling on the Accuracy of Food Image Classification

to categorize food from photographs. Feature-based models have been used to classify food photographs for many years. The frequent qualities that have been used to categorize food photos are SCD, EFD, GFD, and LBP. CNNs in particular has been utilized in contemporary literature to categorize food-related pictures. In Fig 1, the two model diagrams show the technique used to ensemble models in our experiments. In the left model, we are averaging the parameters of the ensembled model, and in the right one, we are only ensembling the models. The graph below shows the accuracy of different model combinations using different techniques.

II. Related Works

In [1] deep feature extraction, feature concatenation and support vector machine (SVM) classifiers were used to efficiently classify food images. The optimal deep feature sequence for classifying food images was determined by concatenating the features of size 4096 that are taken from the fc6 and fc7 layers in various combinations. They used three different datasets: Food-5k, Food-11 and Food-101.

In [2], they used a Convolutional neural network to classify food images. Convolutional neural networks are particularly effective at extracting spatial characteristics from images, making them a great choice for problems involving image categorization. They used the Food-11 dataset and also used a pre-trained Inception V3 convolutional neural network model to classify food images.

In [3], they made a system named WorldFoodMap, which captures food images from various social media platforms that people share. They used a Convolutional neural network for food image classification. Their system allows categorizing trendy food all around the globe. They used a k-NN “k-nearest neighbors” machine learning classifier to make predictions.

In [4], they used a GoogleNet model, which is based on the Convolutional neural network and ran experiments on food/non-food classification and food recognition. They

created their two own datasets where the images were collected from various resources such as existing image datasets, social media, and imaging devices. The classification of food and non-food exhibits a high accuracy of 99.2%, and the recognition of food categories exhibit an accuracy of 83.6%.

Model soups improve accuracy over the best individual model when performing a large, random hyperparameter search for fine-tuning a CLIP ViT-B/32 model on ImageNet. Model soup is a concept of averaging weights of multiple fine-tuned models improve accuracy without increasing inference time [5]. The resulting ViT-G model reached a new state of the art, achieving 90.94% top-1 accuracy on ImageNet.

In [6], they designed a convolutional neural network and provided a food dataset collection to distinguish the groups. In order to achieve this, the two pre-trained models, Caffenet and Alexnet, were adjusted, and a comparable structure was trained using the dataset. Images of food were created using the datasets Food-11, FooDD, Food100, and web archives. They have got 70.12% with their proposed approach, 80.51% with Caffenet and 82.07% accuracy with Alexnet.

In [7], they used the ensemble technique to show the superiority in classification accuracy with many experiments. They used two algorithms i.e. adaboost and bagging. In the context of machine learning, an ensemble can be broadly characterized as a system built with a number of independent models operating in parallel, whose outputs are then integrated using a decision fusion technique to generate a single solution to a specific problem.

In [8], Kawano et al. suggested a system for smartphones for real-time user-aided visual food recognition using CNNs, leveraging HOG characteristics and color histograms. s The recognition process is performed repeatedly and continuously. In the experiments, they achieved a 79.2% classification rate for the top 5 category candidates for a 100-category food dataset with the ground-truth bounding boxes when we used HOG and color patches with the Fisher Vector coding as image features.

In [9] a brief description of the methods of the ensemble was given. The basic ensemble method is Bayesian average aging. Using a weighted vote of the classifiers' predictions, ensemble methods categorize fresh data through learning algorithms that build a group of classifiers. The researcher examines various techniques and illustrates how ensembles can frequently outperform any single classifier. It also shows why Adaboost does not overfit rapidly.

In [10], Food datasets from FCD (FOOD-101 and Caltech-256) and RagusaDS were used in experiments, and accurate results for the two datasets were 94.97% and 99.01%, respectively. A pre-trained DCNN model for food recognition was improved. DCNN was very suitable for large-scale image data since it takes only 0.03 seconds to classify one food photo with GPU.

III. Methodology

During image classification problems, we often tend to try different types of models to predict our images. Different models have different qualities; hence they learn different types of features of the given image. Some models may produce a better result, but some may not. But how about combining the models to produce a more accurate result? Here comes model ensembling. A technique to combine models and produce a greater than the single model. Another thing is that, while training a model during each iteration, the model might learn new things and forget other things. What if we could retain the old learned features and add new ones? This is where we are averaging the parameters of our model. The main idea is to retrain the old information and add new ones to improve accuracy. Our goal is to combine these techniques to achieve a greater result that can be applied to any model or model. By this, we can use those models to find a greater result whenever we train different models to classify our image.

A. *Alexnet*

AlexNet is a convolutional neural network that was trained on the ImageNet dataset. It was developed by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton and published in the paper "ImageNet Classification with Deep Convolutional Neural Networks" in 2012. The model consists of eight layers, including five convolutional layers and three fully connected layers. The image dataset contains images 227*227 with RGB channels. So the input tensor is [227,227,3]. Here we have 11 different data classes. So there will be 11 predictions for each class. The AlexNet model consists of eight layers. The first convolutional layer has 96 filters with a size of 11x11 and a stride of 4. It also has a padding of 0. The number of trainable parameters in this layer is (11x11x3+1)x96 = 34,944. The output volume has a size of 55x55x96.

$$\textbf{Output} = \textbf{Convolution(Input, Filter)} + \textbf{Bias} \quad (1)$$

The Max Pooling layer 1 has a pool size of 3x3 and a stride of 2. It does not have any trainable parameters. The output volume has a size of 27x27x96.

$$\textbf{Output} = \textbf{MaxPool(Input)} \quad (2)$$

The 2nd convolutional layer has 256 filters with a size of 5x5 and a stride of 1. It also has a padding of 2. The number of trainable parameters in this layer is (5x5x96+1)*256 = 614,656. The output volume has a size of 27x27x256. Then again, a Max Pooling layer has a pool size of 3x3 and a stride of 2, and the output volume has a size of 13x13x256. The 3rd convolutional layer, it has 384 filters of size 3x3 and a stride of 1, and a padding of 1. The number of trainable parameters in this layer is (3x3x256+1)*384 = 885,120. So the output volume is 13x13x384. 384 filters with a size of 3x3 and a stride of 1, and a padding of 1 is used in the 4th convolutional layer in the model. The number of trainable parameters in the 4th convolutional layer is (3x3x384+1)*384 = 1,327,488. The output volume has a size of 13x13x384. In the 5th convolutional layer,256 filters with a size of 3x3 and a stride of 1 were used. It also has a padding of 1. The number of trainable parameters in this layer is (3x3x384+1)*256 = 884,992 and output size of 13x13x256.Then we apply a max

pooling layer of 3x3 with a stride of 2 and a padding of 0. So output dimension is 6x6x256. Then we flatten this output into one single row vector. Finally, a Fully Connected layer to convert this into 4096x1 row vector and the number of parameters in this layer are (6x6x256)x4096 + 4096 = 37752832. Again another fully connected layer but this time the output size will not change so number of parameters are (4096*4096) + 4096 = 16781312. Lastly another fully connected layer to convert it into our desired class number. Hence the output size is 11x1 and (4096x11) + 11 = 45067 parameters.

$$\textbf{Output = MatrixMultiply(Input, Weights) + Bias} \quad (3)$$

In total there are 58291467 parameters in this model.

B. VGG 16

Convolutional neural network VGG16 is regarded as one of the top computer vision models. The VGG16 model is a convolutional neural network that was trained on the ImageNet dataset and developed by the Visual Geometry Group at the University of Oxford. It was first described in a 2014 publication titled "Very Deep Convolutional Networks for Large-Scale Image Recognition.

The model consists of 16 layers, including 13 convolutional layers and 3 fully connected layers. Each convolutional layer is followed by a rectified linear unit (ReLU) activation function and a max pooling layer. The model was designed to use small 3x3 convolution filters, with a stride of 1 and a max pooling layer with a size of 2x2 and stride of 2. One key point is that, the paddings in the convolutional layers are done in such a way that the input height and width is the same as the output height and width. The Food 11 dataset contains images of size 227x227 with 3 channels. So the input tensor is [227,227,3]. Here we have 11 different data classes. So our output will have score over all the 11 classes.

In the first convolution layer 64 filters were used on the input and the output dimension was 227x227x64, Which means there were (3x3x3)x64 + 64 = 1792 parameters. Again another 64 filters were applied but the output dimension didn't change and the numbers of trainable parameters were (3x3x64)x64 +64 = 36928. After that max pooling was applied changing the output size to 113x113x64. Then again 2 convolution layers were applied each containing 128 filters followed by a max pooling layer. After that the output shape was 56x56x128 and total trainable parameters were (3x3x64)x128 + (3x3x128)x128 + 2x128 = 221440. Now we will apply another 3 convolution layer followed by a max pooling layer and this time each convolution layer will have 256 filter. Total trainable parameters are (3x3x128)x256 + 2x(3x3x256)x256 + 3x256 = 1475328 and output is 28x28x256. Again repeat this step but with 512 filters in each layer. So the output is 14x14x512 and trainable parameters (3x3x256)x512 + 2x(3x3x512)x512 + 3x512 = 5899776. Again we do this step to convert our output into 7x7x512 and total parameters in this layer are 3x(3x3x512)x512 + 3x512 = 7079424.

Now we flatten this output and apply a fully connected layer to turn it into to a row vector of 4096x1 followed by a relu function. So the number of trainable parameters are (7x7x512)x4096 + 4096 = 102764544. Again we apply a fully connected network but this time the output size will not change. So the number of trainable parameters are (4096x4096) + 4096 = 16781312. Lastly another fully connected layer to get scores on our classes. So the final output is 11x1 and number of trainable parameters are (4096x11) + 11 = 45067. Softmax Layer: This layer applies the softmax function to the output of the previous fully connected layer to produce a probability distribution over the 11 class labels. It does not have any trainable parameters. The mathematical formula for this layer is:

$$\textbf{Softmax}(p_i) = \frac{\exp(s_i)}{\sum_j \exp(s_j)} \quad (4)$$

Total trainable parameter in this model is 128995811.

C. Resnet 50

ResNet-50 is a convolutional neural network designed to be trained on the ImageNet dataset. Its architecture consists of five stages, each containing one or more residual blocks. These blocks perform identity mapping, which involves adding the output of the block to its input. This allows the network to learn residual functions, which are the differences between the input and output of the block. The first stage of the network includes a single convolutional layer with 64 filters and a max pooling layer, while the second stage consists of three residual blocks with three convolutional layers each. The third stage contains four residual blocks with four convolutional layers each, the fourth stage contains six residual blocks with six convolutional layers each, and the fifth stage contains three residual blocks with three convolutional layers each. The total number of trainable parameters in ResNet-50 is approximately 25.6 million. The mathematical formulas for the convolutional layers and the identity mapping in the residual blocks are as follows: Convolutional layer:

$$\textbf{Output = Input * Weight + Bias} \quad (5)$$

Identity mapping:

$$\textbf{Output = Input + F(Input)} \quad (6)$$

where F is the function learned by the residual block. Here we have 11 different categories in the data set hence there will be 11 scores in the prediction.

$$\hat{Y} = [\hat{y}_0, \hat{y}_1, ..., \hat{y}_{10}]$$

This vector represents the classification probability for the corresponding data classes. For this model, input dimension will be 227x227x3.

D. *Inception V3*

InceptionV3 is a convolutional neural network that has been trained on the ImageNet dataset and has been shown to attain an accuracy greater than 78.1%. It is used for assisting in image analysis and object detection. As the third edition of Google's Inception Convolutional Neural Network, the design of Inceptionv3 was intended to allow deeper networks while also keeping the number of parameters from growing too large. The Inception V3 model has a total of 48 layers. It has a total of around 23 million parameters and is known for its efficiency in terms of computation and the number of parameters. The mathematical formula for Inception V3 is not a single, straightforward equation, as it is a complex model made up of many layers and operations. However, at a high level, the model uses convolutional and pooling layers to extract features from the input image and then uses fully connected layers to classify the image based on those features. The model takes an input image of size 299x299x3 (width x height x channels). The model outputs an 11-dimensional vector of class scores, where each element in the vector represents the model's predicted probability that the input image belongs to a particular class.

E. *Model Parameter Averaging*

We utilized parameter averaging techniques since we wanted to keep the previously learnt features and then put the new features on top of them. For this technique, we repeatedly trained the same model on our dataset. Then, among those models, choose the best one, and train it a second time for the same amount of iterations. But this time, we averaged the model's parameters between each iteration and the one before it, then we inserted those parameters to the model. By doing this, the model both picks up new features and keeps its existing ones. As an illustration, let's call a model M_A. The best model M_{A_BEST} is now obtained after N epochs of training the model M_A on the dataset. Once more, we train the M_{A_BEST} model for N iterations. We obtain a model M_{A_1} in the first iteration, and we then load a new parameter into the model by averaging the parameters of M_{A_1} and M_{A_BEST}, (M_{A_BEST} + M_{A_1}) / 2. To obtain the best averaged model for this dataset, this approach will be repeated for the remaining iterations.

for key in M_{A_BEST} **do**
M_{A_BEST}[key] = (M_{A_BEST}[key] + M_{A_1}[key]) / 2
end for

The four convolutional models used here are Alexnet, Resnet50, VGG16, and Inception V3. We trained these models for a certain number of epochs and then again trained them but with the parameters averaged.

Ensemble class 1: This is a class which takes in models and ensembles them to produce the output. To ensemble two models this class takes in those two models as parameters and feeds the images to the models. After the models return their individual outputs, the ensemble class averages them to make one output and return it. The size of the output of the ensemble class is the same as the other models. Fig 2. is a code representation of our ensemble class. In the constructor it is taking in four models to create an object of the class and averaging the outputs of those models in the forward function.

```
class MyEnsemble(torch.nn.Module):
    def __init__(self, modelA,modelB, modelC,modelD):
        super(MyEnsemble, self).__init__()
        self.modelA = modelA
        self.modelC = modelC
        self.modelB = modelB
        self.modelD = modelD

    def forward(self, x1,x2, x3,x4):
        x1 = self.modelA(x1)
        x3 = self.modelC(x3)
        x2 = self.modelB(x2)
        x4 = self.modelD(x4)
        x = x1 + x2 + x3 + x4
        x = torch.mul(x, (1.0/4.0))
        return torch.softmax(x, dim=1)
```

Fig. 2. My Ensemble

F. *Traditional Ensembling*

A machine learning technique called ensemble modeling combines the predictions of several trained models to provide predictions that are more accurate than those of any single model. Because numerous forms of errors can be made by separate models, it is hypothesized that the predictions made by combining predictions from various models will be more accurate than those made by any one model. Predictions from different models can be combined in a number of ways, including by computing the mean, median, or mode of the predictions or by training a meta-model to do so. For example, we have three models that are trained to classify a set of images into one of 10 classes, each model will make a prediction for each image, then the predictions will be ensembled and the final prediction will be the class with the highest percentage or score.

Here, in order to ensemble the models, we average their outputs and choose the category with the highest score as our prediction. In Fig 3, we are ensembling 2 models M_A and M_B and our input is X (food image). So the outputs are Y_A = M_A(X) and Y_B = M_B(X). So this class will return Y = (Y_A + Y_B) / 2 and our predicted class/category will be,

$$Output = Max(Y) \tag{7}$$

So the general formula will be:

$$Y = \frac{1}{n}\sum_{i=1}^{n} y_i \tag{8}$$

n being the number of classes ensembled in the model. We applied this method on three types of model combinations, combining 2 models, combining 3 models and combining 4 models.

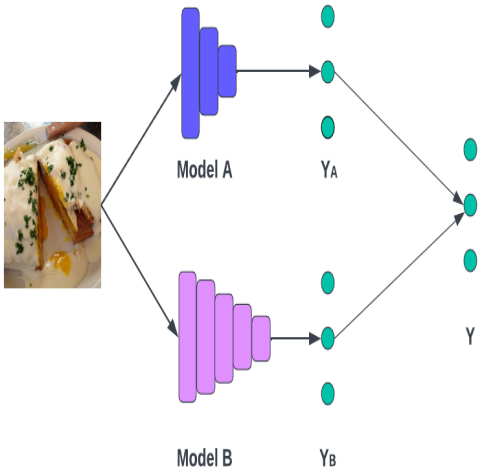


Fig. 3. Traditional Ensembling

G. *Traditional Ensembling Parameter Averaging*

Our four models (Alexnet, Resnet50, VGG16, and Inception V3) have all been trained; we've averaged their model parameters and saved them. In this strategy, we average the model output using a similar ensemble method. But this time, we're utilizing models with averaged parameters rather than those that were regularly trained. Therefore, we must import the previously saved averaged parameters into the models for this strategy to operate. We then averaged the outputs from the models to produce our desired output category.

As an illustration, the average parameters for the models Resnet50 and VGG16 are M_{resnet} and M_{vgg16}. We establish two models, $M_{resnet50_avg}$ and M_{vgg16_avg}, after loading the parameters in the models, and then ensemble them. The models $M_{resnet50_avg}$ and M_{vgg16_avg} generate two outputs, $Y_{resnet50_avg}$ and Y_{vgg16_avg}. Then these models are fed into the MyEnsemble class which sends back a single tensor Y_{avg}. So our output category will be,

$$Output = Max(Y_{AVG}) \tag{9}$$

We applied this method on three types of model combinations, combining 2 models, combining 3 models and combining 4 models.

H. *Traditional Ensembling*

I. *Our Ensemble*

Ensemble class 2: This is another class which also ensembles the given models but in a different manner. Similar to the ensemble class 1 this also takes the in the models as parameters. Instead of averaging the parameter this models concatenates them into one output and a fully connected layer is applied onto the output get our desired number of classes. The size of the concatenated layer will depend on the number of models ensembled. Fig 4. is a code representation of the ensemble class were the constructor is takes in 4 model to build the object and the forward function concatenates the outputs and applies a fully connected layer onto it to get our outputs.

```
class MyEnsemble2(torch.nn.Module):
    def __init__(self, modelA,modelB, modelC,modelD):
        super(MyEnsemble2, self).__init__()
        self.modelA = modelA
        self.modelC = modelC
        self.modelB = modelB
        self.modelD = modelD
        self.classifier = torch.nn.Linear(44, 11)

    def forward(self, x1,x2, x3,x4):
        x1 = self.modelA(x1)
        x3 = self.modelC(x3)
        x2 = self.modelB(x2)
        x4 = self.modelD(x4)
        x = torch.cat((x1,x2, x3,x4), dim=1)
        x = self.classifier(x)
        return x
```

Fig. 4. My Ensemble 2

1) ***Normal Ensemble****:* With this strategy, we attempted to ensemble the models in a unique way before applying the averaging technique to the resulting models.The main idea of this technique was to give our models the images and those models will provide us a tensor of scores on the categories. As a hidden layer of the ensembled model, we will combine those outputs into a single tensor. Then, in order to obtain the appropriate number of classes, we apply a fully connected network. In this method we used the normally trained models for ensembling and trained for certain number of epochs.

In Fig 5, the model M_A and M_B gives us the output Y_A and Y_B for input X. So we created our MyEnsemble2 class using the model M_A and M_B. Then the ensemble class will take the Y_A and Y_B output and concatenate them into a single layer. After that a fully connected network will be applied on that layer to get our 11 class output. Let's call the output $\hat{Y}$ and the category with the maximum score will be the prediction.

$$Y_A = M_A(X) \tag{10}$$

$$Y_B = M_B(X) \tag{11}$$

$$Y_{AB} = (Y_A, Y_B) \tag{12}$$

$$\hat{Y} = W \times Y_{AB} \tag{13}$$

$$Output = max(\hat{Y}) \tag{14}$$

Here (Y_A,Y_B) means the concatenation of outputs and we multiply a weight matrix with it get the $\hat{Y}$. As there were three types of models combination this method was applied on all of them.

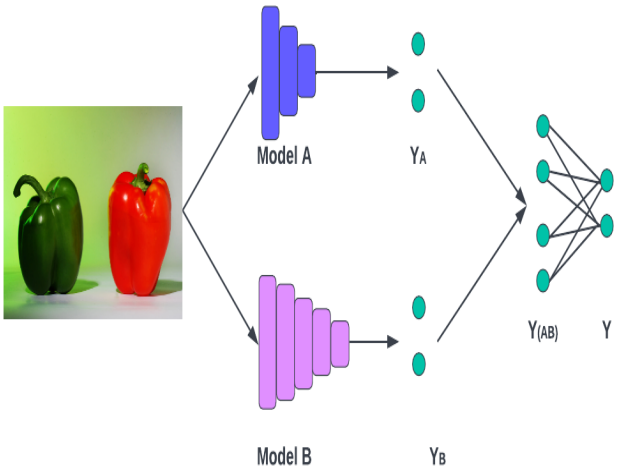


Fig. 5. Normal Ensembling

*2) **Average Ensemble**:* This method follows the same format as the one before. But this time instead of the normally trained models we will be using the models containing average parameters. So to do so we load our averaged parameters into our models and feed those models in the MyEnsemble2 class. The rest of the system is the same.

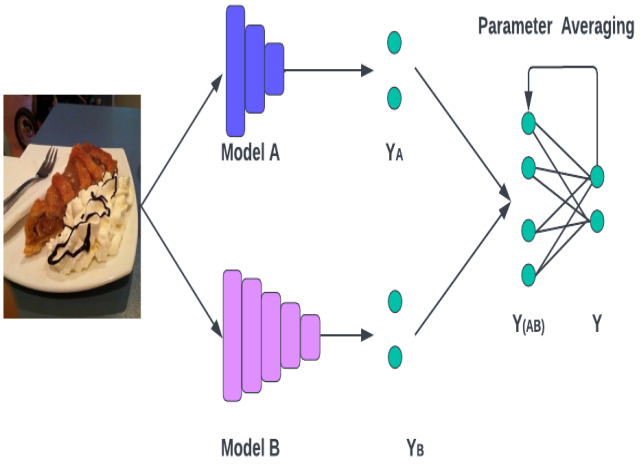


Fig. 6. Normal Ensemble Averaging Parameters

*3) **Normal Ensemble Averaging Parameters**:* This time, we are averaging the Normal Ensembled model's parameters. Therefore, the model is trained conventionally for the first X number of epochs. The parameters from the $X + 1^{th}$ iteration and the parameters from the best model from the previous X epochs are then averaged and loaded into the ensemble model in the $X + 1^{th}$ epoch. For the remaining $X - 1^{th}$ epochs, this procedure will be repeated, and at the end, we will have our best ensemble model with averaged parameters.

If the best ensemble model in the first X epochs is $M_{ensemble_best}$ and the model in the $X + 1^{th}$ epoch is $M_{ensemble_x+1}$ then $M_{ensemble_avg}$,($M_{ensemble_best}$ + $M_{ensemble_x+1}$) / 2 will be loaded into the model and trained again, this will repeat till the end of iteration. Finally $M_{best_ensemble_avg}$, will be the best ensembled model with averaged parameters. In Fig 6. the ensembled model is being trained in epoch iteration and the average of the parameters are loaded back into the model.

*4) **Average Ensemble Averaging Parameter**:* This method is a combination of the Average Ensemble method and Normal Ensemble Averaging Parameters method. Here we ensemble the models contained averaged parameters then we trained them for X epochs conventionally and for the rest X epochs we averaged the parameters according to the Normal Ensemble Averaging Parameters method. The final model will have averaged parameters models ensembled with averaging parameters.

J. Experimental Procedure

For this work, all experiments were run on Google Colab. Google Colab is a cloud computing platform and it provides its own GPU although there is a daily usage limit. The Colab has 12 GB of RAM and an NVIDIA Tesla K80 GPU. To train all of the models, we used the SGD optimizer and a learning rate of 0.001. Our experiment was conducted in three stages.

*1) **Dataset**:* The name of our dataset is the Food-11 dataset. The dataset has 11 categories of food as implied by its name. There are a total of 16643 food images which are divided into three sets: Training set, Validation set, and Evaluation set. The training set has 9866 images, the evaluation contains 3347 images and the validation is made up of 3430 images. The 11 different categories of food in this dataset are Bread, Dairy products, Dessert, Egg, Fried food, Meat, Noodles/Pasta, Rice, Seafood, Soup, and Vegetable/Fruit. The dataset was given labels from 0 to 10, with 0 denoting Bread and 10 being Vegetable/Fruit, and so on. We decided to reduce the size of the validation set to 991 images and it now comprises 991 images or roughly 90 images per category. The rest of the images were shifted to the training set which increased the size of the set to 12305 images. Fig 7 is a representation of the different classes in our Food-11 dataset.
Link : *https://www.kaggle.com/datasets/vermaavi/food11*

*2) **Image Processing**:* Before providing the images into our models, we first pre-processed the images to ensure that the images have the correct dimensions to fit into our model and the models can train properly. We also normalized the images so that the data in the images are distributed properly. For Alexnet, Resnet50 and VGG16 the dimension of the images were 227*227*3 and for Inception V3 it was 299*299*3. After



Fig. 7. Dataset

that, we randomly flipped those images horizontally so that the models can learn from those as well and lastly normalized them with a mean of 0.5 and standard deviation of 0.5 across all channels.

*3) **Regular Model Training**:* We took four pre-trained CNN models: Alexnet, VGG16, Resnet50 and Inception V3. Alexnet was trained over 100 epochs; VGG16 81 epochs, Resnet50 was trained over 38 epochs and 32 epochs for Inception V3 using SGD optimizer with Cross Entropy Loss Function.

*4) **Parameter Averaging Model**:* As discussed in the methodology section, to improve a machine learning model, we used parameter averaging, which involves training the same model multiple times and selecting the best one. We then train this model again, averaging the parameters at each iteration with the previous iteration's parameters and inserting them into the model. This allows the model to both retain its existing features and learn new ones.

We used four convolutional models: Alexnet, Resnet50, VGG16, and Inception V3. For the first 50 epochs, we trained Alexnet normally and averaged the remaining 50 epochs. For the VGG16 model, we trained it normally for 40 epochs and averaged the remaining 41 epochs. For the Inception V3 model, we trained it normally for 16 epochs and averaged the remaining 16 epochs. For the Resnet 50 model, we trained it normally for 19 epochs and averaged the remaining 19 epochs.

*5) **Traditional Ensemble**:* For traditional ensembling, we found the outputs from our image inputs over 11 classes for each model. These outputs were then averaged to find the main output for our ensemble. The category with the highest score was taken as our prediction. The traditional ensembling technique was applied on three types of model combinations: combining 2 models, combining 3 models and combining 4 models. Each ensemble was trained over 10 epochs.

*6) **Our Ensembling Method**:*

*a) **Normal Ensemble**:* We tried a different approach to ensemble modeling by having the models provide us with a tensor of scores on the categories for each image, and then combining those outputs into a single tensor as a hidden layer of the ensembled model. We used a fully connected network to obtain the appropriate number of classes from this tensor. We used the normally trained models for ensembling and trained the ensemble model for 10 epochs.

*b) **Average Ensemble**:* It uses models with averaged parameters instead of normally trained models. We load the averaged parameters into our models and feed them into the MyEnsemble2 class and then repeat the same process as we did in Normal Ensemble over 10 epochs.

*c) **Normal Ensemble Averaging Parameters**:* This method involves averaging the parameters of the normal ensemble model. The model is trained normally for the first five epochs, and then the parameters from that iteration are averaged with the parameters of the best model from the previous five epochs and loaded into the ensemble model in the sixth epoch. This process is repeated for the remaining 4 epochs to obtain the best ensemble model with averaged parameters at the end.

*d) **Average Ensemble Averaging Parameter**:* This method combines the Average Ensemble method and the Normal Ensemble Averaging Parameters method. We first ensemble models with averaged parameters and train them normally for five epochs. For the remaining 5 epochs, we average the parameters of the models according to the Normal Ensemble Averaging Parameters method.

The results from our ensembling methods were compared with the results obtained by traditional ensembling and compared to see which works best. Further details are discussed in the results and analysis section.

IV. Results

The data obtained from training our models were recorded in two different tables. As specified earlier, we used 4 pre-trained models: Alexnet, Resnet50, VGG16, and Inception V3. The results obtained from training the dataset on the individual models were recorded in Table 1.2. Aside from that, we used ensemble learning to train another 11 models. These models were made up of all possible combinations of our initial 4 models which totaled to 11. Six different Ensembles were trained on these models and their results were recorded in Table 1.3

Model Combination	Baseline Ensemble	Average Ensemble	Baseline Ensemble Average	Average Ensemble average	Traditional ensemble	Traditional average ensemble
Alexnet,InceptionV3	93.51	93.99	93.66	94.08	93.51	93.49
Alexnet,Resnet50	93.21	93.87	93.93	94.02	93.51	93.63
InceptionV3,Resnet50	94.95	94.65	94.98	94.32	95.1	95.21
Alexnet,InceptionV3,Resnet50	95.01	94.14	94.26	94.65	93.78	94.8
Alexnet,VGG16	92.2	90.88	91.6	91.6	93.36	93.6
InceptionV3,VGG16	93.15	93.63	94.44	94.56	94.8	94.68
Alexnet,InceptionV3,VGG16	93.18	94.05	92.85	92.85	94.2	94.68
Resnet50,VGG16	93.75	93.45	93.69	93.6	95.01	94.95
Alexnet,Resnet50,VGG16	93.51	94.08	93.96	93.69	93.69	94.74
InceptionV3,Resnet50,VGG16	93.75	94.23	94.44	94.65	94.92	95.24
Alexnet,InceptionV3,Resnet50,VGG16	94.35	94.17	94.17	94.83	94.17	95.15

table1.3

Model	Normal Training	Parameter Average
Alexnet	86.58	87.09
InceptionV3	93.99	94.29
Resnet50	94.89	94.71
VGG16	92.08	93.48

table1.2

Analyzing Table 1.2: For the 'Alexnet' model, the accuracy for the validation dataset was 85.67%, and for the evaluation dataset was 86.58%. The accuracy for the validation dataset increased to 86.07%, and the evaluation dataset to 87.09%.when the weight of the model was averaged

As seen in Table 1.2, The 'InceptionV3' model had an accuracy of 93.23% for the validation dataset and 93.99% for the evaluation dataset. If we average the weight of the model, the accuracy slightly increases to 93.54% for the validation dataset and 94.29% for the evaluation dataset.

The 'Resnet50' model had an accuracy of 93.84% for the validation dataset and 94.89% for the evaluation dataset. The accuracy slightly decreases to 93.64% for the validation dataset and 94.71% for the evaluation dataset once it has been averaged.

The 'VGG16' model had an accuracy of 91.62% for the validation dataset and 92.08% for the evaluation dataset. The weighted average of the model shows accuracy slightly increasing to 92.23% for the validation dataset and 92.48% for the evaluation dataset.

After recording and analyzing our results from Table 1.2, we proceeded to make all possible combinations of the four pre-train models. We made six types of ensembles: Baseline Ensemble, Average Ensemble, Baseline Ensemble Average, Average Ensemble Average, Traditional Ensemble, and Traditional Average Ensemble and ran them on 11 different model combinations. The results were recorded in Table 1.3:

To analyze the data of Table 1.3, we shall compare the attributes of data in pairs. That is: Baseline Ensemble results will be compared with that of Baseline Ensemble Average; Average Ensemble with Average Ensemble Average and Traditional Ensemble with Traditional Ensemble Average.

In table 1.3, while comparing the accuracy scores of Baseline Ensemble and Baseline Ensemble Average on our model combinations, the highest accuracy score was obtained to be 95.01% from the Baseline Ensemble of Alexnet, Inception V3 and Resnet 50. Overall out of the 11 pairs of results compared, Baseline Ensemble Average results were higher in 6 out of the 11 combinations.

During the comparison of the accuracy scores of Average Ensemble and Average Ensemble Average, the highest accuracy score was obtained to be 94.83% from the Baseline Ensemble of Alexnet, Inception V3 Resnet 50 and VGG16. Overall out of the 11 pairs of results compared, Average Ensemble Average results were higher in 8 out of the 11 combinations.

Finally we compared the accuracy scores of Traditional Ensemble and Traditional Average Ensemble from Table 1.3. The highest accuracy score was 95.24% in the Traditional Average Ensemble of Inception V3, Resnet50 and VGG16. Traditional Average Ensemble results were also higher in 8 out of the 11 combinations.

From the above result, we see that the best accuracy for the validation dataset was 94.75% and the best accuracy for the evaluation dataset was 95.24%.

V. Conclusion

In this project, we applied parameter averaging and model ensembling based on CNN architecture on the task of food image classification. We used the Food-11 dataset containing 11 categories of food. For the experiment, we made 15 combinations from 4 CNN models (Alexnet, VGG16, Resnet50, and InceptionV3) and conducted parameters averaging and models ensemble techniques resulting in **74** different experiments. The highest accuracy obtained from the experimental results was **95.24%** in model combination of Resnet50, VGG16, and InceptionV3 via average ensembling with parameter averaging technique. As a future direction, we aim to carry out these experiments in a more balanced dataset. As well as introducing more CNN models and new averaging techniques.

References

[1] Şengür, A., Akbulut, Y. and Budak, Ü., 2019, September. Food image classification with deep features. In 2019 international artificial intelligence and data processing symposium (IDAP) (pp. 1-6). Ieee.

[2] Islam, M.T., Siddique, B.N.K., Rahman, S. and Jabid, T., 2018, October. Food image classification with convolutional neural network. In 2018 International Conference on Intelligent Informatics and Biomedical Sciences (ICIIBMS) (Vol. 3, pp. 257-262). IEEE.

[3] Amato, G., Bolettieri, P., Monteiro de Lira, V., Muntean, C.I., Perego, R. and Renso, C., 2017, August. Social media image recognition for food trend analysis. In Proceedings of the 40th international ACM SIGIR conference on research and development in information retrieval (pp. 1333-1336).

[4] Singla, A., Yuan, L. and Ebrahimi, T., 2016, October. Food/non-food image classification and food categorization using pre-trained googlenet model. In Proceedings of the 2nd International Workshop on Multimedia Assisted Dietary Management (pp. 3-11).

[5] Wortsman, M., Ilharco, G., Gadre, S.Y., Roelofs, R., Gontijo-Lopes, R., Morcos, A.S., Namkoong, H., Farhadi, A., Carmon, Y., Kornblith, S. and Schmidt, L., 2022, June. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In International Conference on Machine Learning (pp. 23965-23998). PMLR.

[6] Özsert Yiğit, G. and Özyildirim, B.M., 2018. Comparison of convolutional neural network models for food image classification. Journal of Information and Telecommunication, 2(3), pp.347-357.

[7] Huang, F., Xie, G. and Xiao, R. (2009) "Research on ensemble learning," 2009 International Conference on Artificial Intelligence and Computational Intelligence [Preprint].

[8] Kawano, Y. and Yanai, K., 2015. Foodcam: A real-time food recognition system on a smartphone. Multimedia Tools and Applications, 74(14), pp.5263-5287.

[9] Dietterich, T.G., 2000, June. Ensemble methods in machine learning. In International workshop on multiple classifier systems (pp. 1-15). Springer, Berlin, Heidelberg.

[10] Yanai, K. and Kawano, Y., 2015, June. Food image recognition using deep convolutional network with pre-training and fine-tuning. In 2015 IEEE International Conference on Multimedia Expo Workshops (ICMEW) (pp. 1-6). IEEE.